CITI BIKE REBALANCING EXPLORER · PROJECT REPORT

# Station imbalance as a barrier to mode choice

What the tool shows, what the data can and cannot support, and where it could go next.

---

**Live app:** nyc-citibike-rebalancing.vercel.app

**Code and data:** github.com/rezataeb/nyc-citibike-rebalancing

**Prepared:** November 2026 · Independent project · public data only. Not an official NYC DOT or Lyft product.

**About the numbers:** every figure in this report is read from the project's committed data files at build time (`report/build_report.py`), not typed in by hand.

## 1. Summary

An empty or full bike-share station is more than an operations nuisance. A rider who finds no bike, or no dock, may stop treating bike share as a real option. This tool shows where and when that happens, how reliable stations have actually been, and what a fixed truck fleet could realistically fix.

- **Stations were unusable 13.1% of the time.** That is the share of 2,922,701 station observations with 0 bikes or 0 docks, from Jul 13, 2026 to Oct 31, 2026. Full stations outnumbered empty ones about 2.8 to 1.
- **Bigger stations are associated with better reliability.** After accounting for demand, stations with 40+ docks were unusable about 2.2 percentage points less often than stations with under 20 docks. This is a pattern in observational data, not proof of cause.
- **A simple baseline beat the machine-learning models.** Over 12 walk-forward test months, a seasonal-naive forecast had the lowest error (MAE 1.92 bikes/day, vs. 2.03 for the guarded model). The dashboard is honest about this rather than presenting the more complex model as better. A stricter forward-only benchmark agrees hour by hour, but for weekday morning-peak totals a recency-weighted forecast (Holt-Winters) is +16.6% better (Section 4.4).
- **Trucks alone cannot close the gap.** In the fleet simulator, 10 trucks fully rebalanced 155 of 1,039 deficit stations (14.9%) under a fixed morning baseline.
- **Some questions cannot be answered with public data.** A dock-capacity "what if" sandbox was investigated and closed, because station capacity over time was never recorded (Section 5.2).

## 2. The problem

Bike share only works as a transportation mode if it is reliably there. When a station is empty, a rider cannot start a trip. When it is full, a rider cannot end one, and has to ride on to find a dock. Both failures push people toward other modes. If they happen often at the stations serving a neighborhood, bike share stops being a real choice for the people who live there. That makes station imbalance an access question, not only a logistics one.

The City's Citi Bike contract, as summarized in the NYC Comptroller's review *Riding Forward*, requires stations not to sit completely full or empty, with the penalty applying only when adjacent stations are also unavailable. The tool is built for the oversight and planning side of that relationship. Lyft, the operator, decides how its own trucks run; this tool helps a reviewer at NYC DOT see where imbalance concentrates, how reliable stations have been, who is affected, and what a given fleet could realistically cover.

> **WHO IT IS FOR**
>
> NYC DOT Bike Share & Shared Mobility reviewers, and anyone checking reliability claims against public data. It is a static web page with precomputed data: no account, no backend, and a $0 stack.

## 3. Data and methods

### 3.1 Sources

| Layer | Source | Coverage used here |
|---|---|---|
| Trips | Citi Bike S3 trip archives | 12 months, Oct 2025 – Sep 2026; 2,523 stations (2,054 with every month covered) |
| Station status | Citi Bike GBFS feed | Logged by a scheduled GitHub Action; 1,222 snapshots, Jul 13, 2026 – Oct 31, 2026, on 98 days. Also read live by the dashboard. |
| NYCHA developments | NYC Open Data `phvi-damg` | 216 records; 300 m proximity |
| Schools | NYC Open Data `wg9x-4ke6` | 1,899 records (2019–2020 locations); 300 m proximity |
| Subway stations | NY State Open Data `i9wp-a4ja` | 2,120 records; 800 m subway-gap distance |
| Weather | Open-Meteo historical archive | Daily temperature and precipitation for 18 fixed zones (a 6 km grid) |

### 3.2 Quality rules (fixed)

- Trips under 60 seconds or over 4 hours are dropped.
- Low-volume stations are excluded from forecasting and typology clustering.
- Offline stations are excluded from failure denominators: a station that is switched off is not counted as "empty" or "full".

### 3.3 Methods, plainly

**Net flow**

Arrivals minus departures per station, by hour, for weekdays and weekends, by season and month. Negative means the station drains; positive means it fills.

**Station typology**

K-means clustering of each station's weekday hourly shape. k = 2 had the best silhouette score (0.41): 694 "commuter core" stations (fill in the morning, drain in the evening) and 1,115 "residential feeder" stations (the reverse). 714 stations had too little traffic to cluster.

**Demand forecast**
Three tiers compared honestly: a seasonal-naive baseline, a spline-based GAM, and a gradient-boosted model with an extrapolation guard that falls back to simpler tiers outside the training temperature range. Validated by 12-fold walk-forward testing (each month held out in turn), not a single split.

**Reliability rate**
Share of online station observations in the GBFS log with 0 bikes or 0 docks. A frequency, not outage hours (see Section 5.1).

**Capacity vs. reliability**
Ordinary least squares: unusable rate ~ capacity + log(1 + demand), across stations.

**Fleet simulator**
Greedy truck routing over the morning deficit/surplus baseline. Each truck carries 20 bikes and makes at most 45 stops. Ten scenarios, 1 to 10 trucks.

**Equity context**
Distance from each station to the nearest NYCHA development, school, and subway station. Of 2,523 stations, 310 are within 300 m of NYCHA, 1,542 within 300 m of a school, and 362 are 800 m or more from a subway station.

## 4. Findings

### 4.1 How often stations were unusable

Across 2,922,701 online station observations, stations had 0 bikes or 0 docks **13.1%** of the time (381,813 observations). 125,202 offline observations were excluded. The window runs from Jul 13, 2026 to Oct 31, 2026: midsummer into fall (Section 5.1).

- **Full is the bigger problem.** 282,919 observations were full and 101,527 were empty, about 2.8 to 1. A full station blocks the end of a trip, which is arguably worse for a rider than an empty one: they are already on a bike with nowhere to leave it.
- **The burden is uneven.** Among 2,507 stations with at least 20 observations, the median station was unusable 10.8% of the time; the worst was unusable 68.4% of the time.

### 4.2 Capacity and reliability

Across 2,438 stations, after controlling for demand, each extra dock is associated with an unusable rate about 0.11 percentage points lower (standard error 0.02). Over a 20-dock difference, that is about 2.2 points.

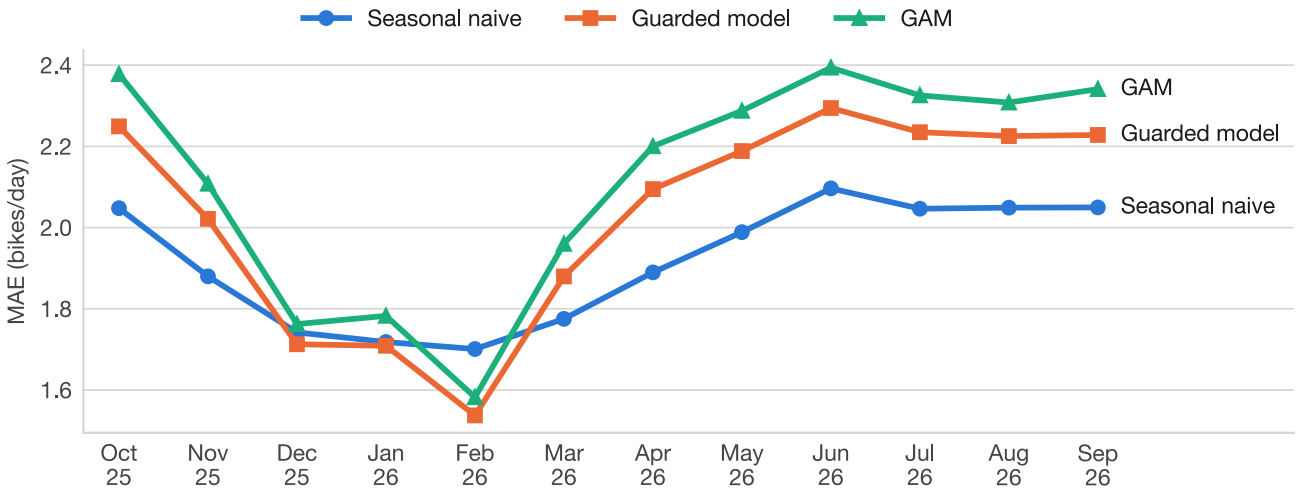
| Station size | Stations | Mean unusable rate |
|---|---|---|
| < 20 docks | 610 | 15.5% |
| 20-39 docks | 1,376 | 13.4% |
| 40+ docks | 452 | 10.1% |

*Raw band means, before the demand control. The regression above is the adjusted result.*

> **PATTERN, NOT PROOF**
>
> This compares different stations at one point in time. It does not show that adding docks to a given station would make it more reliable. Borough and zone are not in the pipeline, so location is not controlled for.

### 4.3 Forecasting: the simple baseline won

Mean absolute error across 12 walk-forward folds: seasonal naive **1.92**, guarded model 2.03, GAM 2.12 bikes/day. Naive's advantage is statistically significant (Wilcoxon signed-rank, p = 0.012 vs. guarded and p = 0.002 vs. GAM). The guarded model did win Dec 2025, Jan 2026, Feb 2026; naive won the other 9 months.




*Forecast error by held-out month. Lower is better.*

The practical reading: for a station's typical flow at a given hour, its own historical average for that hour and day type is hard to beat. The models only pulled ahead in the coldest months. The dashboard therefore shows historical patterns as the main view and labels the model comparison openly, instead of presenting a model forecast as the answer.

### 4.4 A stricter test: forward-only time-series forecasts

The comparison above holds out one month at a time, so a held-out month can learn from later months, and it scores only station-hours that had trips. A second benchmark ( `pipeline/forecast_benchmark.py` )

removes both shortcuts. It uses the 2,054 stations with trips in every month, counts hours without trips as zero, and makes each forecast at the start of a week from earlier data only, for the whole week ahead (38 test weeks from Jan 5, 2026). It compares the same-hour profile (the naive baseline) with last week's values, the mean of the last four weeks, Holt-Winters with a weekly season (alpha 0.02, gamma 0.2, tuned on earlier weeks only), and a gradient-boosted model with lagged values and the day's actual weather.

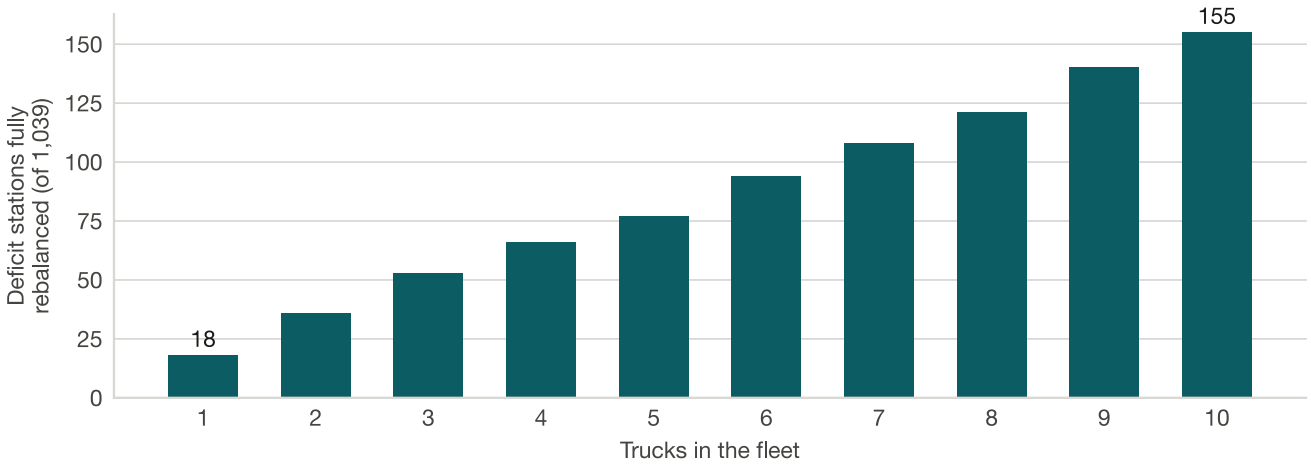
| Method | Hourly | Weekday 6–9 AM total | Daily total |
|---|---|---|---|
| Same-hour profile (naive) | 1.27 | 4.11 | 4.21 |
| Last week | 1.69 (−32.7%) | 4.22 (−2.8%) | 5.75 (−36.6%) |
| Mean of last four weeks | 1.38 (−8.2%) | 3.56 (+13.4%) | 4.59 (−8.9%) |
| Holt-Winters | 1.31 (−2.6%) | 3.43 (+16.6%) | 4.32 (−2.6%) |
| Gradient boosting + weather | 1.25 (+2.2%) | 3.51 (+14.5%) | 4.36 (−3.4%) |

*Mean absolute error in bikes (net flow), with the change against the profile; positive is better.*

**The naive baseline is still the right default, with one exception.** Hour by hour and for daily totals, nothing beats the profile by much: gradient boosting is +2.2% hourly (better in 27 of 38 weeks), and it is given the actual weather, which a real forecast would not have, so that gain is an upper bound. Last week alone is the worst method everywhere. The exception is the **weekday morning-peak total**, the number a rebalancing plan needs: Holt-Winters is +16.6% against the profile (better in 34 of 38 weeks), and even the four-week mean is +13.4%. Recent weeks carry information about the morning peak that a long average blurs. The dashboard does not use this yet (Section 6).

### 4.5 What a truck fleet can cover

The morning baseline flags 1,039 deficit stations and 569 surplus stations. One truck fully rebalances 18 deficit stations; 10 trucks rebalance 155 (14.9%). Every truck in every scenario hits the 45-stop cap, so the binding limit is stops per shift, not a lack of stations needing work.




*Deficit stations fully rebalanced, by fleet size (cumulative).*

For a planner this points beyond trucks: where imbalance is structural and recurs every day, station size and placement may matter as much as rebalancing, which is why the capacity result (4.2) is worth testing further.

## 5. Limitations

These are the real limits of what this tool can claim. Most are also stated in the dashboard or the repository, near the number they affect.

### 5.1 GBFS sampling resolution

The reliability numbers come from 1,222 snapshots taken over 110 days (Jul 13, 2026 – Oct 31, 2026). 98 of those days have data. The median gap between snapshots is 65 minutes. The longest gap, 14.5 days (Aug 30, 2026 – Sep 13, 2026), is when the log file hit GitHub's 100 MB limit and the scheduled job could not save new snapshots until it was fixed.

A correction: an earlier build of these numbers covered only July 13–26 but was labeled "July 13 – September 13". It was found while writing this report and rebuilt from the full log on October 28, 2026. The dashboard now states how many days have data and names the longest gap.

- **A rate, not a duration.** We can say how often a station was found unusable. We cannot say for how many minutes. A 15-minute outage can start and end between two snapshots and never be seen.
- **Not the contract measure.** The contract's 60- and 120-minute thresholds cannot be resolved from snapshots this far apart. Nothing here is a penalty calculation.
- **Summer and fall only.** Winter and spring reliability are not measured yet, and the capacity result in 4.2 rests on the same window.

### 5.2 Dock capacity sandbox (Phase 5): closed, not viable

**What it was.** The original plan named a "dock capacity sandbox": pick a station, change how many docks it has, and see how its risk of running empty or full would change. A planning tool for real capacity decisions.

**What was investigated.** Two things, against the real snapshot log (2,440,748 rows, 2,481 stations, as of September 13, 2026):

1. *Can the log measure outage duration?* Partly. When a station was caught unusable and had another reading within 70 minutes, it was still unusable **67.9%** of the time (135,563 of 199,552). Outages tend to persist. But short outages are missed between polls, so any minutes-based estimate would be biased long.
2. *Is there any record of a station's capacity changing?* No. The logger recorded bikes and docks available, but never the `capacity` field itself. The obvious proxy, bikes + docks, appears to vary at 92% of stations, but that is docks going temporarily out of service, not stations being resized. One checked station's proxy moved between 16 and 20 across 981 snapshots while its true capacity stayed 19. No public third-party archive of capacity history was found either; the Comptroller's own GBFS project has the same gap.

**Decision.** Closed as not viable, not deferred. The input the feature needs, capacity changing at the same station over time, was never collected, and real resizing is too rare to collect enough of in any realistic timeframe. The persistence finding (about 2 in 3 outages still present an hour later) is kept and shown in the dashboard's Live Docks data notes.

**The alternative.** The closeout named a different question that public data *can* answer: do stations that already differ in capacity differ in reliability, network-wide, once demand is held constant? That cross-sectional comparison was later built, and it is the result in Section 4.2. It answers "what is associated with reliability," not "what would happen if this station were resized."

Full write-up: `docs_v2/phase5-closeout.md`.

### 5.3 Live mode caveats

- **An instant reading.** Live Docks shows the GBFS feed at the moment the page loads (refreshed every 60 seconds). "Service availability" and "double-outage" are point-in-time approximations, not the contract's 60-minute measurement.
- **"Adjacent" is this project's definition.** A double-outage is a station with 0 bikes or 0 docks that has another such station within 500 m. The 500 m radius is a project choice, not the contract's, and the dashboard says so.
- **Some stations drop out.** Stations with no live match, a capacity of 0, or not both renting and returning are excluded from live counts, and the excluded count is shown.
- **Capacity exists only in live mode.** Per-station capacity comes from the live `station_information` feed. The capacity analysis in 4.2 therefore pairs today's capacity with a past reliability window. Capacity rarely changes (5.2), so the risk is small but real.

- **The truck list is a fixed baseline.** Priority truck stops come from the historical morning baseline and do not change with live conditions.

### 5.4 Other limitations

- **No borough or zone control.** The capacity analysis controls for demand only.
- **Weather scenarios are elasticities, not a weather model.** They scale historical flow by fitted temperature and precipitation sensitivities. A heat-wave preset was left out because the fit does not support extrapolating that far.
- **School locations are from 2019–2020.** It was the only coordinate-bearing school dataset on NYC Open Data when checked.
- **Truck routing is a sketch.** No real depot locations are public, so each tour starts at the largest surplus station. The 45-stop cap is a rough shift estimate, not a time or distance model.
- **Trips show demand that was served, not demand that was lost.** A rider who gave up at an empty station leaves no trip record, so the trip data undercounts demand exactly where reliability is worst.
- **Associations, not causes.** Typology, equity proximity and the capacity result describe patterns. None of them is a causal estimate.

## 6. Future steps

Ordered by value to a DOT reviewer per unit of effort. None of these is built yet.

1. **Forecast the morning peak with recent weeks.** The benchmark in 4.4 shows a recency-weighted forecast is clearly better for weekday morning-peak totals. Showing that next to the historical pattern, labeled as a forecast, would give a truck plan a better starting number. Low effort: the method is already written and tested.
2. **Roll the baseline year forward.** The data window is set in one file (`pipeline/window.py`) and one command rebuilds everything. Rolling it each spring and fall would allow a first seasonal comparison of reliability, which the logger keeps collecting on the `data-snapshots` branch.
3. **Log the full `station_information` response on every poll.** This is what the Comptroller's project does. It would start a real capacity history going forward, so the gap in 5.2 cannot happen again for any other field.
4. **Poll more often.** Snapshots every 10–15 minutes would let the log approach the contract's 60-minute threshold and estimate outage durations, not only rates. This trades off against repository size and GitHub Actions scheduling delays.
5. **Add borough and community district.** Joining NYC Open Data boundary files would add the missing location control to the capacity analysis and let results be reported by community district.

6. **Cross-check against operator reports.** Compare the measured unusable rates with Citi Bike's published monthly operating reports, to see whether the two tell the same story.
7. **Estimate lost demand.** Combine reliability with trip data to estimate trips that did not happen because a station was empty or full. This would turn reliability into riders affected, which is the number a mode-choice argument needs.
8. **Revisit weather once summer data accumulates.** More hot days in the training data would make a moderate hot-day or heat-wave scenario defensible.

## 7. How AI and LLMs could improve the tool

The core rule: the pipeline computes every number, and a language model only explains, retrieves, or drafts around those numbers. Used that way, an LLM makes the tool easier to use for people who will never open a JSON file. **The first application below is built and live** (the dashboard's "Ask the data" panel); the rest are not built yet.

### 7.1 Useful applications

**Plain-language station briefings (built)**
For any station, a short summary from its own record: its flow pattern, typology, reliability rate and rank, and nearby NYCHA, schools and subway. A council member's office or a community board reads two or three plain sentences instead of a chart. Briefings are written once in the pipeline by Claude Haiku through the Batch API, checked by code (every number must match the station's record; a failed briefing falls back to a template), and saved as JSON, so the site stays static and free to host. Next to them, ready-made questions follow the New York clock ("Where do bikes run out next hour?", answered from the same calendar month of the baseline year) and are computed by code, not written by the model.

**Questions in plain English**
A reviewer asks "Which commuter-core stations were full most often in August?" The model turns that into a structured query (filters, sort, limit) that ordinary code runs against the existing data files. The answer is a table, and the query is shown next to it, so the reviewer can check what was asked.

**Draft oversight memos**
Each time the data refreshes, draft a one-page memo on what changed: stations whose reliability moved most, new worst-performing stations, and how the fleet picture shifted. A person reviews and edits it before it goes anywhere.

**Linking text feedback to measured reliability**
If DOT has free-text rider feedback or community board testimony about bike share, an LLM could tag each comment with a station and a problem type (empty, full, broken dock). Joining those tags to the measured rates would show whether complaints track real outages or point to problems the data misses. This depends on a text source that has not been identified yet.

**Explaining changes**
When a station's rate jumps between refreshes, an assistant could gather the context the pipeline already has (weather, nearby stations, typology) and suggest likely reasons, clearly marked as hypotheses for a person to check.

## 7.2 Guardrails

- **No invented numbers.** Every number in generated text must appear in the input record. An automated check compares them and rejects the text if any number is new (built for the briefings), the same way this report refuses to build with an unfilled placeholder.
- **Show the source.** Every generated answer names the data file and window it came from.
- **Say "I don't know".** When the data does not cover a question, for example outage duration (5.1), the system says so instead of guessing.
- **Test before trusting.** A fixed set of questions with known answers, re-run whenever the prompts or the model change.
- **Keep the cost at zero for viewers.** Run the model in the pipeline and publish its output as static files. No API key in the browser, and no per-visit cost.

How AI was used to build this project is a separate topic, covered in the project's AI-collaboration retrospective.

---

Public data: Citi Bike GBFS and trip data, NYC Open Data, NY State Open Data, Open-Meteo. Rebuild this report with `python3 -m report.build_report`.